

Autoencoder: Frutas

Benítez De La Portilla Rodrigo Antonio

A01771433@tec.mx

Abstract: Este estudio presenta la implementación y evaluación de dos modelos de Inteligencia Artificial diseñados para comprimir y posteriormente reconstruir imágenes, utilizando el conjunto de datos **images**. Ambos modelos emplean una arquitectura compuesta por un codificador (encoder), que reduce las dimensiones de las imágenes, y un decodificador (decoder), que las reconstruye. El primer modelo se basa en una arquitectura **densa**, mientras que el segundo incorpora una arquitectura **U-Net con Skip Connections** para mejorar el procesamiento espacial.

Los resultados muestran que el modelo U-Net logra una mejor generalización, evidenciada por la cercanía entre las curvas de pérdida de entrenamiento y validación, así como por una convergencia más rápida y estable. En cuanto al modelo denso, se observa una tendencia al sobreajuste a partir de la época 60, con una divergencia creciente entre las pérdidas de entrenamiento y validación.

Además, el uso de una función de pérdida compuesta (**MSE + SSIM**) en el modelo U-Net permite preservar mejor la estructura de las imágenes reconstruidas, lo que se traduce en una mayor fidelidad perceptual. Estos hallazgos sugieren que la incorporación de Skip Connections y métricas estructurales en la función de pérdida mejora significativamente el desempeño de los autoencoders en tareas de reconstrucción de imágenes.

Keywords: Inteligencia Artificial, Encoder, Decoder, Autoencoder, Dimensiones, Arquitectura, Densa, U-net, Skip Connection.

1. INTRODUCCIÓN

En los últimos años, el procesamiento de imágenes mediante Inteligencia Artificial ha transformado áreas como la medicina, la seguridad y el entretenimiento, permitiendo desde diagnósticos automatizados hasta mejoras en la calidad visual de contenidos digitales. Sin embargo, la necesidad de almacenar y transmitir grandes volúmenes de datos ha

impulsado el desarrollo de técnicas más eficientes para comprimir imágenes sin perder su esencia visual. Esta idea de compresión no es nueva: desde los primeros métodos de codificación en telecomunicaciones hasta los algoritmos de compresión digital como JPEG, la búsqueda por representar imágenes de forma compacta ha evolucionado hacia enfoques cada vez más inteligentes.

El flujo de trabajo incluye una normalización basada en estadísticas del conjunto de entrenamiento y una partición 70/30 en entrenamiento y validación. Se evalúan dos enfoques complementarios: un autoencoder con arquitectura densa, y otro con la arquitectura u-net y skip connections. El desempeño se mide con **MSE**, **Binary cross-entropy** y **MSE+SSIM** respectivamente y se visualizan tanto las imágenes reales como las imágenes reconstruidas

El objetivo es construir una línea base clara, reproducible y explicable para pronóstico de precios del oro, que sirva como punto de partida para iteraciones futuras. Entre las extensiones recomendadas se incluyen validación temporal (para evitar fuga de información), incorporación de rezagos y ventanas móviles, y variables exógenas (índice del dólar, tasas, inflación, volatilidad implícita). Asimismo, se sugiere contrastar regularización, modelos de árboles y enfoques específicos de series temporales.

En conjunto, este proyecto conecta la tradición histórica de la inversión con herramientas modernas de análisis, mostrando cómo métodos lineales y ensambles pueden apoyar la toma de decisiones en mercados financieros sensibles a la información y al tiempo.

2. DESCRIPCIÓN DEL DATASET

El dataset ***images*** contiene una colección de imágenes digitales de frutas, organizadas en carpetas según su tipo (por ejemplo, apple, banana, grape, etc.). Esta estructura facilita la clasificación supervisada y permite aplicar técnicas de aprendizaje automático orientadas al reconocimiento visual.

2a.- Formato de organización.

Cada carpeta representa una clase, y contiene múltiples imágenes correspondientes a esa fruta. Esta disposición es compatible con flujos de trabajo típicos en visión computacional.

2b.- Características de las imágenes.

Las imágenes presentan una resolución variable, la cual se estandariza durante la etapa de preprocesamiento para asegurar uniformidad en el conjunto de datos. Además, muestran variaciones en ángulos, condiciones de iluminación y escalas, lo que contribuye a una mayor capacidad de generalización por parte del modelo.

2c.- Tamaño del conjunto.

El conjunto completo está compuesto por **298 imágenes distribuidas en 9 clases**, cada una correspondiente a un tipo específico de fruta. Esta organización por categorías permite aplicar técnicas de clasificación supervisada y facilita el análisis comparativo entre clases durante el entrenamiento y evaluación de modelos.

Las clases son:

- **Mango**

- **Orange**
- **Strawberry**
- **Cherry**
- **Chickoo**
- **Grapes**
- **Kiwi**
- **Apple**
- **Banana**

3. TRANSFORMACIÓN DE IMÁGENES

La transformación de imágenes consiste en prepararla para un posterior análisis. Se pueden distinguir tres tipos de transformaciones:

- Transformaciones que son consecuencia de la aplicación de una función sobre el valor de intensidad de cada píxel.
- Transformaciones en las que los píxeles vecinos intervienen en la misma con distintas finalidades, tales como eliminar el ruido presente en la imagen para así suavizarla o con el fin de extraer bordes.
- Transformaciones geométricas, las cuales modifican las coordenadas espaciales de la imagen, ofreciendo aspectos de esta bajo diferentes resoluciones, siendo en ocasiones necesario modificar también los valores de intensidad de los píxeles de la imagen original. []

3a.- Normalización de imágenes.

Primero, las imágenes se redimensionan al tamaño estándar de 28×28 píxeles. En caso de que las imágenes estén en formato RGB, se convierten a escala de grises para reducir la dimensionalidad y facilitar el procesamiento. Posteriormente, los valores de cada píxel se transforman al tipo de dato float32, lo cual mejora la compatibilidad y estabilidad numérica durante el entrenamiento del modelo. Finalmente, se aplica una normalización dividiendo cada valor entre 255, con el objetivo de ajustar el rango de intensidad de los píxeles al intervalo [0, 1].

4b.- División del dataset en conjuntos.

Tras la transformación de las imágenes, el conjunto se segmentó en dos particiones mediante slicing secuencial en una proporción 70%, 30%, preservando el orden cronológico original. Al ser un algoritmo no supervisado, esta decisión facilita aprender representaciones comprimidas.

El 70% inicial se usa para ajustar los parámetros del modelo; el 30% siguiente se reserva para validación, donde se monitorea el desempeño fuera de entrenamiento y se calibran hiperparámetros (por ejemplo, tasa de aprendizaje y número de épocas) sin contaminar la evaluación final.

Este procedimiento aporta beneficios significativos al modelo, ya que mejora la calidad de la reconstrucción de las imágenes. Además, contribuye a reducir el riesgo de sobreajuste mediante el uso de un canal de validación que permite detener o

ajustar el entrenamiento en el momento adecuado.

Asimismo, garantiza la coherencia temporal del problema, lo cual resulta esencial para minimizar la pérdida de información durante el proceso de reconstrucción.

4. CONSTRUCCIÓN Y EVALUACIÓN DEL MODELO

4a.- Selección de arquitectura encoder.

La arquitectura propuesta es de tipo **densa**. En primer lugar, se utiliza una capa **Flatten** para transformar la imagen de dos dimensiones (28×28) en un vector unidimensional, formato requerido por las capas densas. A continuación, la información pasa por cuatro capas densas totalmente conectadas (**fully connected**), lo que aporta simplicidad al modelo y le permite aprender patrones globales en lugar de locales, como ocurre en las redes convolucionales.

Esta elección nos proporciona un baseline sólido, a partir del cual podremos comparar y desarrollar modelos más robustos y especializados.

4b.- Selección de arquitectura decoder.

La arquitectura del decoder comienza con una capa densa que recibe como entrada el vector latente de dimensión reducida (`encoded_dim`).

A partir de ahí, la información se expande progresivamente mediante tres capas

densas totalmente conectadas, que incrementan la dimensionalidad de 128 a 512. Posteriormente, una capa densa con activación **sigmoid** genera una salida de 784 valores, correspondientes a los píxeles de una imagen de 28×28 .

Finalmente, la capa **Reshape** reorganiza este vector en una matriz bidimensional, reconstruyendo la imagen original.

Este diseño permite que el decoder aprenda a mapear representaciones compactas hacia imágenes completas, recuperando patrones globales y ofreciendo una reconstrucción aproximada de la entrada.

4c.- Functional API autoencoder.

La API funcional es una forma de crear modelos más dinámicos que con Sequential; La API funcional puede manejar modelos con topología no lineal, modelos con capas compartidas y modelos con múltiples entradas o salidas.

Se basa en la idea de que un modelo de aprendizaje profundo suele ser un gráfico acíclico dirigido (DAG) de capas. La API funcional es un conjunto de herramientas para construir gráficos de capas. []

La arquitectura del autoencoder se construye integrando el encoder y el decoder en un único modelo. La entrada es una imagen de 28×28 píxeles, que primero pasa por el encoder para obtener una representación latente comprimida. Posteriormente, el decoder utiliza esa representación para reconstruir la imagen original en el mismo formato (28×28).

La función Model de Keras define este flujo de extremo a extremo, desde la entrada hasta la salida reconstruida.

4d.- Función de pérdida.

La función de pérdida elegida es Binary Cross-Entropy, función de pérdida utilizada en problemas de clasificación binaria. Cuantifica la diferencia entre las etiquetas de clase reales (0 o 1) y las probabilidades predichas por el modelo. Cuanto menor sea el valor de la entropía cruzada binaria, mejor se ajustarán las predicciones del modelo a las etiquetas reales. La fórmula es la siguiente: []

$$-\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Ecuación 1. Fórmula de Binary Cross Entropy.

4e.- Optimizador.

Se empleó el optimizador Adam, ampliamente reconocido en el ámbito del aprendizaje profundo por su capacidad para gestionar eficientemente un elevado número de parámetros. Este algoritmo se caracteriza por su rapidez de convergencia, estabilidad en el entrenamiento y naturaleza adaptativa, lo que reduce significativamente la necesidad de realizar ajustes manuales extensivos en los hiperparámetros.

4f.- Métricas.

Aunado a esto para evaluar el rendimiento del modelo se usan dos métricas

principales como binary cross-entropy y mean squared error (MSE) como métrica complementaria, con el propósito de evaluar la magnitud promedio del error de reconstrucción y proporcionar una medida más intuitiva de la calidad de las imágenes generadas, pues mse ofrece una perspectiva geométrica del error al cuantificar directamente la distancia cuadrática entre la imagen original y la reconstruida. Esta característica permite obtener una visión global del desempeño del modelo, favoreciendo la interpretación comparativa de los resultados y facilitando el análisis de la estabilidad del entrenamiento. En conjunto, ambas métricas aportan información complementaria: mientras la binary cross-entropy captura la fidelidad probabilística de la reconstrucción, el MSE refleja la magnitud absoluta de las diferencias, enriqueciendo así la evaluación integral del autoencoder.

4g.- Entrenamiento y Validación.

Para evaluar y potencialmente mejorar la capacidad reconstructiva de nuestro modelo autoencoder, dividimos el conjunto de datos en dos segmentos: entrenamiento (70%) y validación (30%). Esta división es fundamental en el aprendizaje automático, ya que utilizar los mismos datos para entrenar y evaluar el modelo puede provocar un sesgo significativo, donde el algoritmo memoriza los ejemplos en lugar de aprender patrones generalizables, resultando en reconstrucciones pobres para nuevas imágenes.

El conjunto de entrenamiento se utiliza para optimizar los parámetros de nuestro modelo. Durante este proceso, aplicamos el optimizador de Adam para minimizar iterativamente el error cuadrático medio (MSE) entre las imágenes de entrada y las imágenes de salida, ajustando los parámetros en cada época.

El conjunto de validación permanece completamente aislado hasta que el entrenamiento finaliza. Solo entonces lo utilizamos para obtener una evaluación imparcial y definitiva del rendimiento del modelo, calculando tanto el MSE como el binary cross-entropy (BCE) para determinar la precisión y exactitud de nuestras reconstrucciones.

Nuestro enfoque de división del dataset, busca mantener un equilibrio óptimo entre los conjuntos para detectar y prevenir el sobreajuste. En cada época del entrenamiento, registramos el error de entrenamiento, permitiéndonos monitorizar visualmente la evolución de ambos errores en una gráfica temporal. Si el modelo comenzara a memorizar los datos de entrenamiento en lugar de aprender patrones generalizables, observamos una divergencia entre estas curvas, el error de entrenamiento continuaría disminuyendo mientras el error de prueba aumentaría o se estancaría.

El monitoreo constante de estas métricas en los dos conjuntos es crucial para nuestro modelo de reconstrucción. Un rendimiento desequilibrado indicaría que nuestro modelo podría fallar al reconstruir la

imagen. Las visualizaciones que implementamos, especialmente la comparación de pérdida entre los dos conjuntos, nos proporcionan una herramienta efectiva para evaluar si estamos logrando un equilibrio adecuado entre aprender los patrones en las imágenes y mantener la capacidad de reconstruirla con los patrones aprendidos.

4h.- Overfitting.

Al entrenar nuestro modelo de autoencoder para imágenes, identificamos tres escenarios posibles que afectan su capacidad predictiva: subajuste, ajuste adecuado y sobreajuste.

El sobreajuste (overfitting) ocurre cuando el modelo se ajusta excesivamente a los datos de entrenamiento, perdiendo capacidad de generalización frente a datos no vistos. En el caso de autoencoders, este fenómeno se manifiesta cuando el modelo reconstruye con alta precisión las imágenes del conjunto de entrenamiento, pero falla al hacerlo con las de validación.

Podemos prevenirlo mediante la penalización de pesos, que evita que los parámetros crezcan demasiado y se ajusten al ruido del entrenamiento.

También se puede apagar un porcentaje de neuronas durante el entrenamiento con el método Dropout(), que le da fuerza al modelo a aprender representaciones más robustas.

Cómo tercera opción tenemos Early Stopping, para detener el entrenamiento cuando la pérdida de validación deja de

mejorar y evita que el modelo continúe ajustándose innecesariamente a los datos de entretenimiento.

Finalmente hay más opciones como reducir la complejidad del modelo, aumentar los datos o una validación cruzada.

5. VISUALIZACIÓN DE RESULTADOS DEL CASO BASE

Después de pasar por el entrenamiento y prueba y realizar las pruebas pertinentes, los resultados muestran que el modelo presenta un caso de overfitting con los siguientes valores:

Entrenamiento:

- BCE: 0.4970
- MSE: 0.0148

Validación:

- BCE: 0.6179
- MSE: 0.0579

Los valores obtenidos para las métricas de pérdida reflejan un comportamiento característico de sobreajuste. En particular, la **binary cross-entropy** en entrenamiento fue de **0.4970**, mientras que en validación ascendió a **0.6179**, lo que indica una discrepancia significativa en la capacidad de reconstrucción entre ambos conjuntos. De manera similar, el **mean squared error** pasó de **0.0148** en entrenamiento a **0.0579** en validación, evidenciando una pérdida de precisión en las reconstrucciones fuera del conjunto de entrenamiento. Estas diferencias sugieren que el modelo ha capturado patrones

específicos de los datos vistos, pero no logra generalizar adecuadamente, lo que se traduce en una alta varianza y una limitada capacidad de adaptación a nuevos ejemplos.

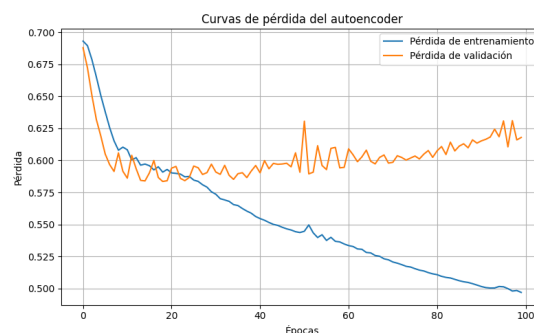


Figura 1. Curvas de error durante entrenamiento y validación.

La Figura 1, muestra las curvas de pérdida del autoencoder durante 100 épocas de entrenamiento. Se observa una disminución sostenida en la pérdida de entrenamiento, lo que indica que el modelo logra ajustar sus parámetros para minimizar el error en los datos vistos. Sin embargo, la pérdida de validación presenta fluctuaciones notables y una tendencia menos estable, lo que sugiere una capacidad limitada de generalización.

En este contexto, el modelo ha aprendido a reconstruir con precisión las imágenes del conjunto de entrenamiento, pero no logra replicar ese nivel de fidelidad en el conjunto de validación. Esto puede atribuirse a una arquitectura con alta capacidad de representación, ausencia de regularización explícita, o una tasa de aprendizaje que favorece ajustes rápidos y específicos. La divergencia entre las curvas de pérdida y las métricas obtenidas sugiere una alta varianza, donde el modelo se adapta excesivamente a los patrones

locales del conjunto de entrenamiento sin capturar adecuadamente la estructura general del conjunto de datos.

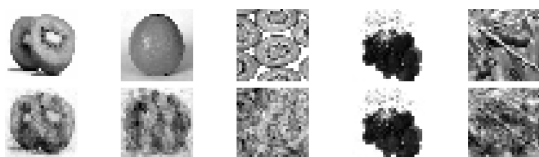


Figura 2. Autoencoder en entrenamiento.

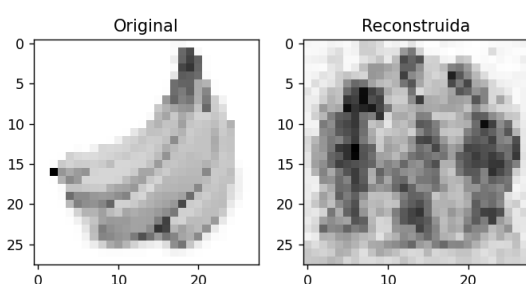


Figura 3. Autoencoder en prueba.

En la primera imagen, correspondiente al entrenamiento, se observa que el modelo logra reconstrucciones con alto nivel de detalle, conservando las formas, texturas y estructuras esenciales de los frutos representados. Esto sugiere que el autoencoder ha aprendido representaciones latentes efectivas para los datos vistos, lo cual es consistente con las métricas obtenidas.

Sin embargo, en la segunda imagen, correspondiente a una prueba con datos no vistos, se evidencia una pérdida notable de fidelidad en la reconstrucción.

6. IMPLEMENTACIÓN DE MEJORA

Dentro de esta parte se presenta la implementación del modelo con una

arquitectura convolucional, utilizando tensorflow, seaborn, numpy, y matplotlib. Con el fin de mejorar las reconstrucciones de las imágenes, aplicando técnicas de preprocesamiento y entrenamiento de modelos, además de analizar los resultados obtenidos.

6a.- Carga y preprocesamiento de datos.

El proceso comenzó cargando el dataset **images**, seguido de un re-escalamiento y garantizar la integridad del conjunto de imágenes.

Este método permite estandarizar los datos al normalizar los valores de los píxeles en las imágenes. De manera predeterminada, dichos valores se encuentran en el rango $[0, 255]$. Al aplicar **rescale=1./255**, cada píxel se divide entre 255 y se transforma al rango $[0, 1]$. Esta normalización resulta fundamental, ya que facilita el entrenamiento de las redes neuronales, contribuyendo a que el modelo aprenda con mayor rapidez y mantenga una mayor estabilidad durante el proceso de optimización.

6b.- División del dataset en conjuntos.

Tras el re-escalamiento de las imágenes, el conjunto se dividió en dos particiones utilizando **ImageDataGenerator**, con una proporción de 70% para entrenamiento y 30% para validación, manteniendo el orden cronológico original. Esta elección resulta equilibrada, ya que asigna una mayor carga al entrenamiento lo cual es esencial para que el modelo aprenda de manera efectiva sin descuidar la

validación, que garantiza una evaluación confiable del desempeño.

6c.- Creación del modelo.

Se implementó un modelo de autoencoder utilizando una arquitectura U-Net con skip connections para la reconstrucción de imágenes. El modelo fue configurado con una función de pérdida combinada (MSE + SSIM) y optimizador Adam, empleando un tamaño de entrada de 224x224 píxeles y un esquema de entrenamiento con early stopping y reducción adaptativa de la tasa de aprendizaje para mejorar la estabilidad y evitar el sobreajuste.

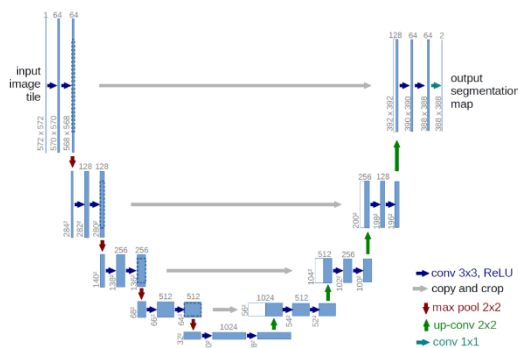


Figura 4. Arquitectura U-net.

La arquitectura U-Net emplea bloques de codificación y decodificación conectados mediante skip connections, lo que permite preservar detalles espaciales importantes durante la reconstrucción. El flujo de trabajo seguido fue el siguiente:

- **Preprocesamiento:** Las imágenes se normalizaron al rango $[0, 1]$ y se dividieron en conjuntos de entrenamiento (70%) y validación (30%) utilizando ImageDataGenerator.

- **Entrenamiento:** El autoencoder fue entrenado durante un máximo de 50 épocas, monitorizando la pérdida de validación para aplicar early stopping y ajustar la tasa de aprendizaje.
- **Reconstrucción:** El modelo toma imágenes de entrada y genera reconstrucciones, permitiendo evaluar la capacidad de compresión y recuperación de información visual.
- **Evaluación:** Se utilizaron métricas estándar como el error cuadrático medio (MSE), el error absoluto medio (MAE) y el índice de similitud estructural (SSIM) para comparar las imágenes originales y reconstruidas en los conjuntos de entrenamiento y validación.

Gracias a la arquitectura U-Net y las técnicas de regularización empleadas, el modelo logra una reconstrucción más precisa y estable, reduciendo la pérdida de información y mejorando la calidad visual respecto a autoencoders convencionales. Esto permite verificar la capacidad de generalización del modelo y minimizar el sobreajuste en tareas de reconstrucción de imágenes.

6d.- Métricas.

La función de pérdida implementada para el autoencoder U-Net combina estratégicamente dos métricas complementarias: el Error Cuadrático Medio (MSE) y el Índice de Similitud Estructural (SSIM). Esta combinación se diseñó específicamente para superar las limitaciones inherentes de usar una sola

métrica de pérdida en tareas de reconstrucción de imágenes.

El MSE, que representa el 70% del peso total de la función de pérdida, calcula la diferencia al cuadrado entre cada píxel de la imagen original y la reconstruida, promediando posteriormente todas estas diferencias.

Para compensar esta deficiencia, se incorporó la pérdida basada en SSIM, que constituye el 30% restante del peso total. El SSIM evalúa la similitud estructural entre imágenes considerando tres componentes fundamentales: luminancia (brillo global), contraste (rango dinámico) y estructura (patrones y texturas).

6e.- Resultados para el modelo de U-net.

Después de entrenar y validar el modelo, los resultados obtenidos fueron los siguientes:

- Entrenamiento:
 - Pérdida: 0.07
 - MAE: 0.06
- Validación:
 - Pérdida: 0.08
 - MAE: 0.11

Los resultados obtenidos durante el entrenamiento del autoencoder U-Net revelan un comportamiento general saludable del modelo, aunque con algunas áreas que requieren atención para optimizar su desempeño.

La proximidad entre la pérdida de entrenamiento y validación es un indicador positivo que sugiere que el modelo ha logrado un balance adecuado entre el aprendizaje de patrones de los datos de

entrenamiento y su capacidad de generalización a datos no vistos.

Sin embargo, al analizar el Error Absoluto Medio (MAE), se observa una discrepancia más pronunciada entre entrenamiento y validación. Esta divergencia, aunque no extrema, sugiere que el modelo está comenzando a mostrar signos incipientes de sobreajuste en términos de error absoluto de reconstrucción de píxeles

7. VISUALIZACIÓN DE RESULTADOS DEL MODELO U-NET

Para entender mejor los resultados se presentan las imágenes generadas por nuestro modelo u-net

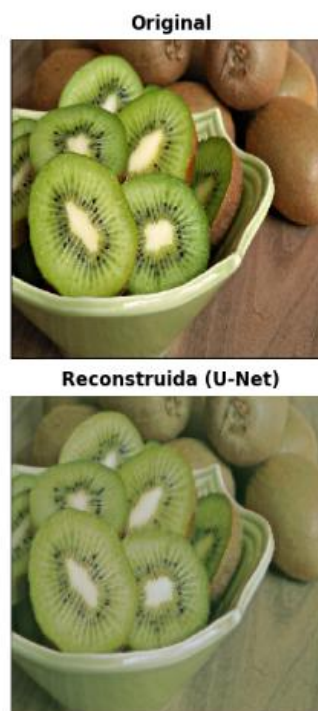


Figura 5. Imagen real vs reconstruida.

En estas imágenes podemos observar cómo nuestra reconstrucción, no se aleja mucho de la imagen real, esto es muy buen resultado pues el rendimiento es muy bueno.

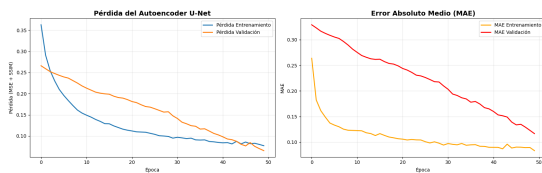
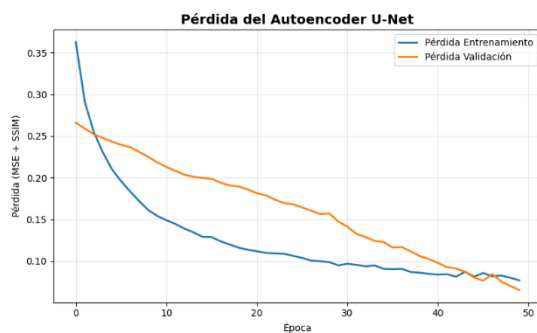
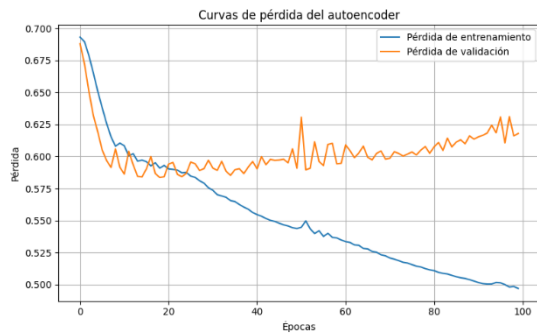


Figura 6. Comparación de métricas.

En la gráfica izquierda se observa la función de pérdida combinada (MSE + SSIM), donde tanto la curva de entrenamiento (azul) como la de validación (naranja) descienden de manera pronunciada desde aproximadamente 0.36 hasta estabilizarse alrededor de 0.08, mostrando una convergencia similar entre ambas que indica un aprendizaje efectivo sin signos severos de sobreajuste. La gráfica derecha muestra el Error Absoluto Medio (MAE), donde la curva de entrenamiento (naranja) disminuye de 0.26 a 0.09, mientras que la curva de validación (rojo) desciende más lentamente desde 0.33 hasta aproximadamente 0.12, evidenciando una brecha moderada entre ambas métricas que sugiere la presencia de un ligero overfitting, aunque el modelo logra generalizar razonablemente bien en datos no vistos durante el entrenamiento.

8. COMPARACIÓN DE LOS MODELOS

A continuación, se presenta una comparación entre los dos modelos



Las curvas de pérdida presentadas permiten comparar el comportamiento de dos arquitecturas de autoencoder durante el entrenamiento: una red densa tradicional y una variante basada en U-Net con conexiones de salto. En el caso del modelo denso, se observa una disminución progresiva de la pérdida de entrenamiento a lo largo de 100 épocas; sin embargo, la pérdida de validación comienza a aumentar a partir de la época 60, lo que indica un claro sobreajuste. Esta divergencia entre ambas curvas sugiere que el modelo está memorizando los datos de entrenamiento sin lograr una adecuada generalización. Por el contrario, el autoencoder U-Net muestra un

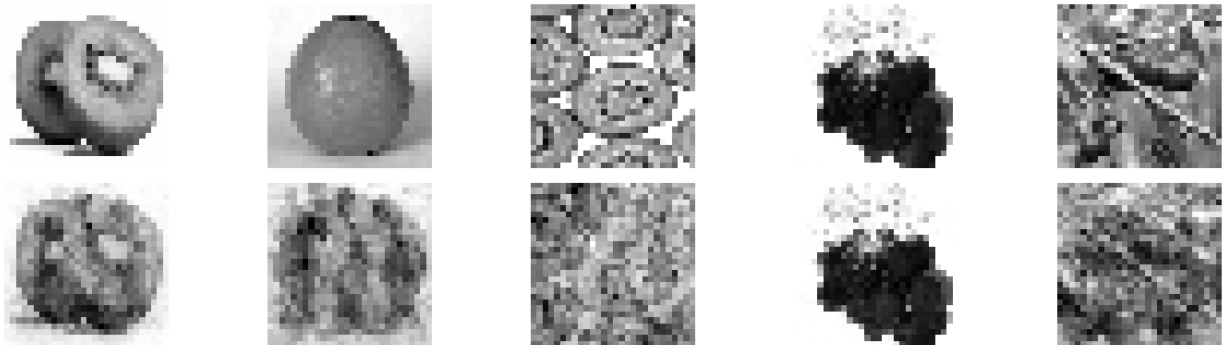
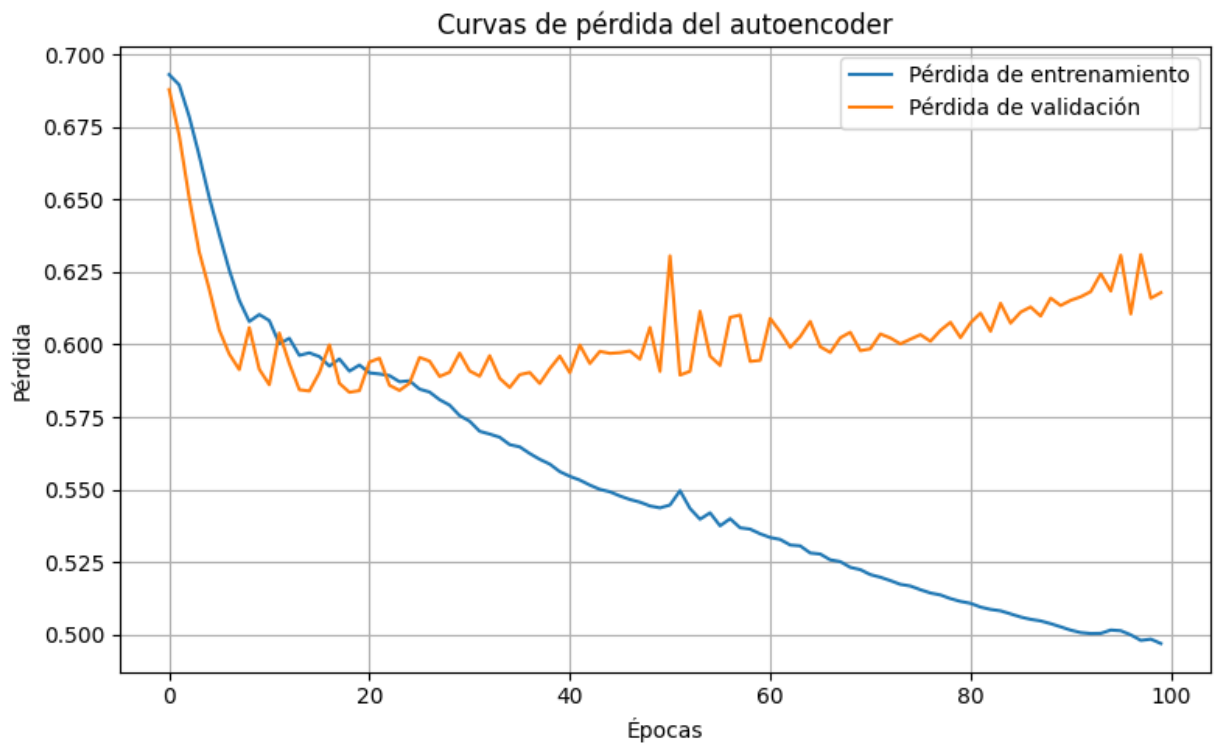
utilizados: Denso y el modelo de U-net. Esta comparación se centra en las métricas de cada modelo como binary cross-entropy mse+ssim, etc.

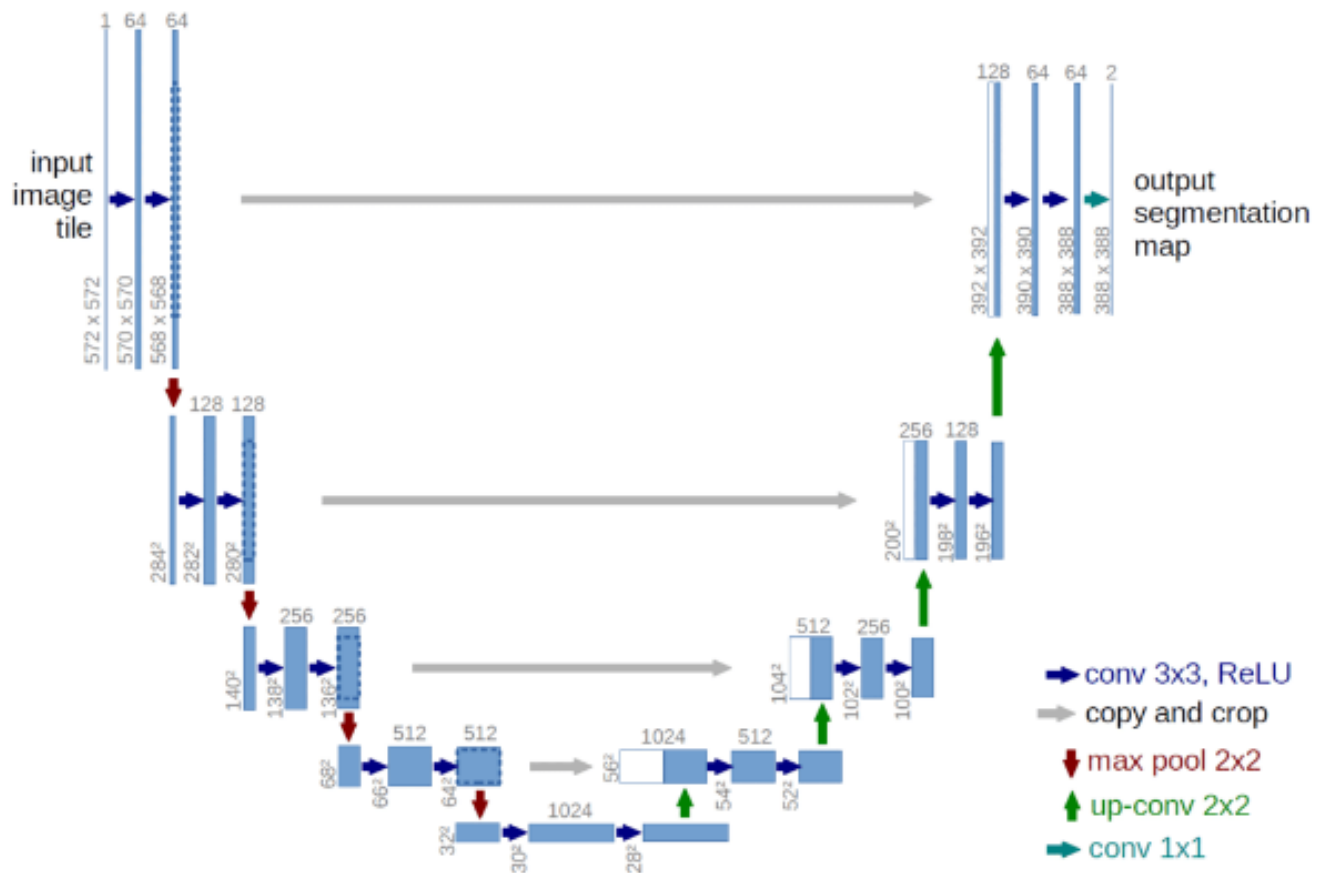
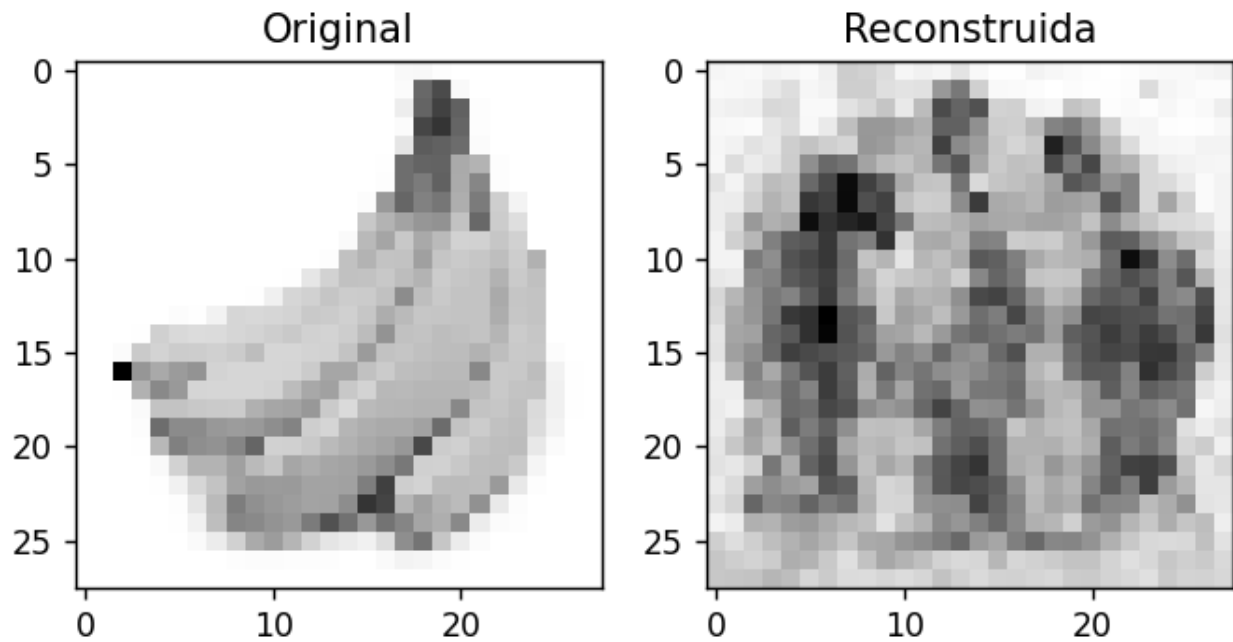
comportamiento más estable y eficiente: tanto la pérdida de entrenamiento como la de validación disminuyen de forma consistente durante las 50 épocas, manteniéndose cercanas entre sí. Este patrón refleja una mejor capacidad de generalización, atribuible en parte a las conexiones de salto que permiten preservar información espacial relevante durante la reconstrucción. Además, el modelo U-Net emplea una función de pérdida compuesta (MSE + SSIM), lo que favorece una reconstrucción más precisa y perceptualmente coherente, al considerar tanto el error pixel a pixel como la similitud estructural. En conjunto, los resultados sugieren que la arquitectura U-Net no solo converge más rápido, sino que también ofrece una representación latente más robusta y una salida de mayor calidad en comparación con el modelo denso.

REFERENCIAS

1. likebupt. (2024, September). Normalización de datos: referencia de componente - Azure Machine Learning. Microsoft.com. <https://learn.microsoft.com/es-es/azure/machine-learning/component-reference/normalize-data?view=azureml-api-2>
2. Mohamad. (2017, March 9). MSE – Error Cuadrático Medio. Centro de Ayuda. <https://support.numxl.com/hc/es/articles/115001223423-MSE-Error-Cuadr%C3%A1tico-Medio>
3. Na8. (2017, December 12). Qué es overfitting y underfitting y cómo solucionarlo | Aprende Machine Learning. Aprendemachinelearning.com. <https://www.aprendemachinelearning.com/que-es-overfitting-y-underfitting-y-como-solucionarlo/#:~:text=Tal%20vez%20se%20pueda%20traducir,conocimiento%20que%20pretendemos%20que%20adquieran.>
4. *Conceptos y métodos en visión por computador.* (2016). <https://dialnet.unirioja.es/servlet/a>
5. La API funcional “Keras” en TensorFlow. (2023). TensorFlow. <https://www.tensorflow.org/guide/keras/functional?hl=es-419>
6. GeeksforGeeks. (2024, May 27). *Binary Cross Entropy/Log Loss for Binary Classification.* GeeksforGeeks. <https://www.geeksforgeeks.org/deep-learning/binary-cross-entropy-log-loss-for-binary-classification/>

ANEXOS





Original



Reconstruida (U-Net)

